Deep Learning

MAI 642, DSC 515, ECE 661

Fall 2026

Instructor: Prof. Theocharis Theocharides

Teaching Assistant: Michalis Kontos

Assignment 3

Assignment Date: 14 Oct 2026

Due Date: 28 Oct 2026

Instructions: This assignment includes 2 exercises. You need to submit both your code (if you are using Colab, download the notebook) and a report. The report should include the required plots as well as your answers to the questions. Please submit all of this in one compressed (ZIP) file, named "Assignment3_Yourname" in the corresponding Teams assignment. Ensure to provide clear code, detailed explanations, and visualizations where applicable.

Exercise 1: Recurrent Neural Networks (RNNs)

1. Explain what Recurrent Neural Networks (RNNs) are and how they differ from feedforward neural networks.
2. Describe the vanishing gradient problem in RNNs. Why is it a problem and how can it affect training?
3. Discuss at least two application areas where RNNs have proven to be effective.
4. Implement a simple RNN to perform sentiment analysis on the IMDB movie reviews dataset. Train the model and evaluate its performance.

IMDB Movie Reviews Dataset:

The IMDB movie reviews dataset is a set of 50,000 reviews from the Internet Movie Database (IMDB). These reviews have been labeled by sentiment (positive/negative). Positive reviews are those with a score of 7 or more out of 10, and negative reviews have a score of less than 4 out of 10. Neutral reviews, scoring between 4 and 7, are not included in the dataset. Each review is preprocessed and encoded as a list of word indexes (integers). Words are ranked by how often they occur (in the training set) and only the 10,000 most frequent words are kept. Each word is given an integer index reflecting its frequency rank.

Your task is to build a model that can read in the numerical representation of movie reviews and predict the sentiment of the text (positive or negative).

Expected Input/Output of the Network:

Input: Sequences of integers representing words in a movie review. Each integer maps to a word in the model's internal dictionary.

Output: A binary prediction: **0** for a negative sentiment or **1** for a positive sentiment.

TensorFlow example of IMDB dataset

```
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence

# Load the dataset
# Only considering the top 10,000 most frequently occurring
words
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=10000)

# Pad the data so that all sequences have the same length
x_train = sequence.pad_sequences(x_train, maxlen=500)
x_test = sequence.pad_sequences(x_test, maxlen=500)
print('Train set shape:', x_train.shape)
print('Test set shape:', x_test.shape)
```

PyTorch has no built-in equivalent. You can keep Keras only for loading the dataset and use PyTorch for padding and tensor conversion

```
import torch
import torch.nn.functional as F
from tensorflow.keras.datasets import imdb

# Load the dataset, keeping the top 10,000 most frequent words
(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=10000)

def pad_sequences(sequences, maxlen=500):
    padded = []

    for sequence in sequences:
        # Keep the last maxlen tokens, matching Keras's
default truncation
        tokens = torch.tensor(sequence[-maxlen:],
dtype=torch.long)

        # Add zeros on the left, matching Keras's default
padding
        tokens = F.pad(tokens, (maxlen - len(tokens), 0),
value=0)
        padded.append(tokens)

    return torch.stack(padded)
```

```
x_train = pad_sequences(x_train, maxlen=500)
x_test = pad_sequences(x_test, maxlen=500)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print("Train set shape:", x_train.shape)
print("Test set shape:", x_test.shape)
```

Your goal is to create an RNN that takes these integer sequences as input and predicts the sentiment of the review. A typical RNN model for this task would have the following layers:

- **Embedding Layer:** This layer converts positive integers (indices) into dense vectors of fixed size. It is critical because it will learn the embeddings of words in a space that groups similar words closer together, capturing semantic meaning.
- **SimpleRNN Layer:** A layer that processes sequences of embeddings with RNN cells. SimpleRNN layers are capable of capturing the sequence nature of language, which is fundamental in sentiment analysis, by maintaining a 'memory' of previous inputs.
- **Dense Output Layer:** A fully connected layer with a *sigmoid* activation function, which will output the probability that the review is positive.

You are of course free to use and experiment with any kind of RNN architecture.

Exercise 2: Long Short-Term Memory Networks (LSTM)

1. Explain how Long Short-Term Memory Networks (LSTM) address the vanishing gradient problem.
2. Provide examples of application areas where LSTM networks have demonstrated superior performance compared to standard RNNs.
5. Implement a simple LSTM network for time series prediction using the Reuters Newswire dataset. Train the model and evaluate its performance.

Reuters Newswire Dataset:

The Reuters Newswire dataset is a collection of documents that were collected and labeled in the 1980s for the purpose of developing and evaluating text categorization systems. The version included in Keras has been preprocessed and is a subset of the full dataset.

The dataset is commonly used in text classification and natural language processing tasks. It consists of 11,228 newswires from Reuters, labeled over 46 topics. Each newswire is encoded as a sequence of word indexes (integers). Words are indexed by overall frequency in the dataset, so the integer 1 corresponds to the most frequent word (e.g.: "the"), 2 to the second most frequent word, and so on. Each article is tagged with a single label that corresponds to a topic.

Expected Input/Output of the Network:

Input: Sequences of integers representing words in a document. Each integer maps to a word in the model's internal dictionary.

Output: A classification from the 46 possible topics.

TensorFlow example of Reuters dataset

```
from tensorflow.keras.datasets import reuters
from tensorflow.keras.preprocessing.sequence import
pad_sequences

# Load the dataset
# Only considering the top 10,000 most frequently occurring
words
(x_train, y_train), (x_test, y_test) =
reuters.load_data(num_words=10000)

# Pad the data so that all sequences have the same length
x_train = pad_sequences(x_train, maxlen=20)
x_test = pad_sequences(x_test, maxlen=20)
print('Train set shape:', x_train.shape)
print('Test set shape:', x_test.shape)
```

Again, keeping Keras **only for loading Reuters**, the equivalent **PyTorch** preprocessing is:

```
import torch
import torch.nn.functional as F
from tensorflow.keras.datasets import reuters

# Load the dataset, keeping the top 10,000 most frequent words
(x_train, y_train), (x_test, y_test) =
reuters.load_data(num_words=10000)

def pad_sequences(sequences, maxlen=20):
    padded = []

    for sequence in sequences:
        # Keep the last maxlen tokens (Keras's default
truncation)
        tokens = torch.tensor(sequence[-maxlen:],
dtype=torch.long)

        # Pad with zeros on the left (Keras's default padding)
        tokens = F.pad(tokens, (maxlen - len(tokens), 0),
value=0)
        padded.append(tokens)

    return torch.stack(padded)

x_train = pad_sequences(x_train, maxlen=20)
x_test = pad_sequences(x_test, maxlen=20)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print("Train set shape:", x_train.shape)
print("Test set shape:", x_test.shape)
```

Your goal is to create an LSTM network that takes sequences of integers representing words as input and predicts the topic category of the newswire. A typical LSTM model for this task would have the following layers:

- **Embedding Layer**: Turns positive integers (indexes) into dense vectors of a fixed size. This layer is essential as it learns to encode the input integers into a space where similar words have similar encoding.
- **LSTM Layer**: Processes sequences of word embeddings, capturing temporal dependencies. The LSTM units can remember information for long periods, which is key for understanding the context in the newswire text.
- **Dense Output Layer**: A fully connected layer with a *softmax* activation function. This layer outputs a probability distribution over the 46 different topic categories, predicting the most likely topic for a given newswire.

You are of course free to use and experiment with any kind of LSTM architecture.